

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DSA0305 — NATURAL LANGUAGE PROCESSING

Course Assignment

ASSIGNMENT TITLE

Smart Resource Allocation and Optimization System

Main Topic: Design and Development of a POS Tagging System for Real-Time Text Processing

Class / Section	B.E CSE(AI)
Faculty Name	Dr. K. Anbazhagan
Academic Year / Batch	2026
Course Outcome	CO4 — To design a tag set to be used for statistical processing for real-time applications
Bloom's Taxonomy Level	L3 – Apply, L5 – Evaluate
SDG Mapping	SDG8 – Decent Work and Economic Growth; SDG9 – Industry, Innovation and Infrastructure
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100

TABLE OF CONTENTS

1. Introduction	
3	
1.1 What is Natural Language Processing (NLP)?	3
1.2 What is POS Tagging?	3
1.3 Why POS Tagging is Important	3
1.4 Why Real-Time POS Tagging is Useful	3
1.5 Purpose of this Assignment	
3	
2. Problem Statement	
3	

3. Objectives	4
4. POS Tag Set	4
5. POS Tagging of Given Sentences	5
6. Proposed NLP Processing Pipeline	
.....	6
7. Feature-Based POS Tagging	7
8. Ambiguous Word Analysis: "book"	8
9. Training Dataset	8
10. Machine Learning POS Classifier	9
10.1 Feature Preparation	9
10.2 Feature Extraction	9
10.3 Feature Vectorization	9
10.4 Model Training	9
10.5 Tag Prediction	9
10.6 Output Generation	9
11. Python Implementation	10
11.1 Importing Libraries	10
11.2 Training Data	10
11.3 Tokenization	
.....	10
11.4 Feature Extraction	10
11.5 Feature Vectorization	11
11.6 Model Training	11
11.7 POS Prediction	
.....	11
11.8 Output Generation	11
12. Expected Output	11
13. Algorithm / Pseudocode	
.....	12
14. Evaluation Metrics	12
15. Testing and Performance Analysis	13
16. Challenges	
.....	14
17. Comparison with Modern Contextual Language Models	14
18. Real-World Applications	15
19. Tools and Technologies	
.....	16
20. Advantages	

.....	16	21.
Limitations		
16	22.	Ethical and Social Considerations
.....	17	
23. Conclusion		
17	24.	Student Reflection
.....	18	
25. References		
18		

1. Introduction

1.1 What is Natural Language Processing (NLP)?

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, and generate human language. It combines concepts from linguistics, computer science, and machine learning to allow machines to process text and speech in a way that is meaningful and useful. NLP powers a wide range of applications such as machine translation, chatbots, search engines, and voice assistants.

1.2 What is POS Tagging?

Part-of-Speech (POS) tagging is the process of assigning a grammatical category — such as noun, verb, adjective, or adverb — to each word in a sentence based on its definition and context. POS tagging is one of the foundational tasks in NLP because it provides the syntactic structure required by higher-level tasks such as parsing, named entity recognition, and information extraction.

1.3 Why POS Tagging is Important

POS tagging helps a system understand the grammatical role of each word, which in turn helps resolve ambiguity, build accurate parse trees, and extract meaningful relationships between words. Without POS information, tasks like grammar checking, machine translation, and semantic analysis would be far less accurate.

1.4 Why Real-Time POS Tagging is Useful

Many modern applications, such as chatbots, virtual assistants, live captioning systems, and realtime sentiment analysis tools, must process text as it arrives. A real-time POS tagging system must be fast, memory-efficient, and robust enough to handle unpredictable or informal user input while maintaining acceptable accuracy.

1.5 Purpose of this Assignment

This assignment aims to design, implement, and evaluate a feature-based and machine-learningbased POS tagging system suitable for real-time text processing. It covers the complete pipeline from tokenization to tag prediction, along with an evaluation framework and a comparison with modern contextual language-model-based approaches.

2. Problem Statement

Given a stream of unstructured natural language text, the system must automatically identify the grammatical category (Part-of-Speech) of every word in real time. Many words are lexically ambiguous — the same word form can serve different grammatical roles depending on context (for example, "book" can be a NOUN or a VERB). The proposed solution must therefore extract contextual and linguistic features from the surrounding words, use these features to disambiguate the correct tag, and produce an accurate, tagged output stream efficiently enough for real-time use. The system is evaluated for its scalability across small, medium, and large datasets and compared against modern contextual language-model-based taggers.

3. Objectives

1. To understand the concept and significance of Part-of-Speech tagging in NLP.
2. To design a suitable POS tag set for real-time statistical text processing.
3. To design an end-to-end NLP pipeline for POS tagging, from tokenization to tagged output.
4. To identify and extract relevant contextual and linguistic features for disambiguating ambiguous words.
5. To implement a feature-based POS tagging algorithm using Python.
6. To design and train a machine-learning-based POS tagging classifier.
7. To evaluate the tagging system using standard metrics such as accuracy, precision, recall, and F1-score.
8. To compare the feature-based approach with modern contextual language-model-based approaches.

4. POS Tag Set

The following tag set has been designed for statistical POS tagging of real-time text. Each tag represents a broad grammatical category that is sufficient for practical tagging while remaining simple enough for feature-based classification.

Tag	Category	Meaning	Example
NOUN	Noun	Names a person, place, thing, or idea	flight, book, student, company
PRON	Pronoun	Replaces a noun	She, They, It

VERB	Verb	Expresses an action or state	booked, reads, developed, approved
ADJ	Adjective	Describes or modifies a noun	new, smart, useful
ADV	Adverb	Modifies a verb, adjective, or another adverb	quickly, immediately, every day
DET	Determiner	Introduces a noun and specifies it	The, a, an
PREP	Preposition	Shows relationship between a noun and other words	to, in, on, at
CONJ	Conjunction	Connects words, phrases, or clauses	and, but, or
PROPN	Proper Noun	Specific name of a person, place, or organization	Chennai

Each tag is defined so that it maps directly onto the grammatical categories required for downstream tasks such as chunking and parsing. Determiners and prepositions are kept separate from adjectives and adverbs because they play a structural rather than descriptive role. Proper nouns are separated from common nouns since they typically require capitalization-based features to identify.

5. POS Tagging of Given Sentences

The four given sentences have been manually tagged word-by-word using the tag set defined above.

Sentence 1: "The customer booked a flight to Chennai."

Word	POS Tag
The	DET
customer	NOUN
booked	VERB
a	DET
flight	NOUN
to	PREP

Chennai	PROPN
---------	-------

Sentence 2: "The student reads a book every day."

Word	POS Tag
The	DET
student	NOUN
reads	VERB
a	DET
book	NOUN
every	DET
day	NOUN

Sentence 3: "The company developed a new application."

Word	POS Tag
The	DET
company	NOUN
developed	VERB
a	DET
new	ADJ
application	NOUN

Sentence 4: "The manager quickly approved the project."

Word	POS Tag
The	DET
manager	NOUN
quickly	ADV
approved	VERB
the	DET
project	NOUN

6. Proposed NLP Processing Pipeline

The proposed POS tagging system follows a six-stage pipeline. Each stage transforms the input progressively closer to the final tagged output.

Stage 1: Tokenization

The raw input sentence is split into individual tokens (words and punctuation marks) using whitespace and punctuation-based rules.

Stage 2: Tag-set Design

The fixed set of grammatical categories (defined in Section 4) that the system can assign to each token.

Stage 3: Feature Extraction

For each token, relevant linguistic and contextual features are extracted, such as the previous word, next word, prefixes, suffixes, and capitalization.

Stage 4: Contextual Analysis

The extracted features are analyzed together with neighboring POS tags to resolve ambiguity and understand the token's role within the sentence.

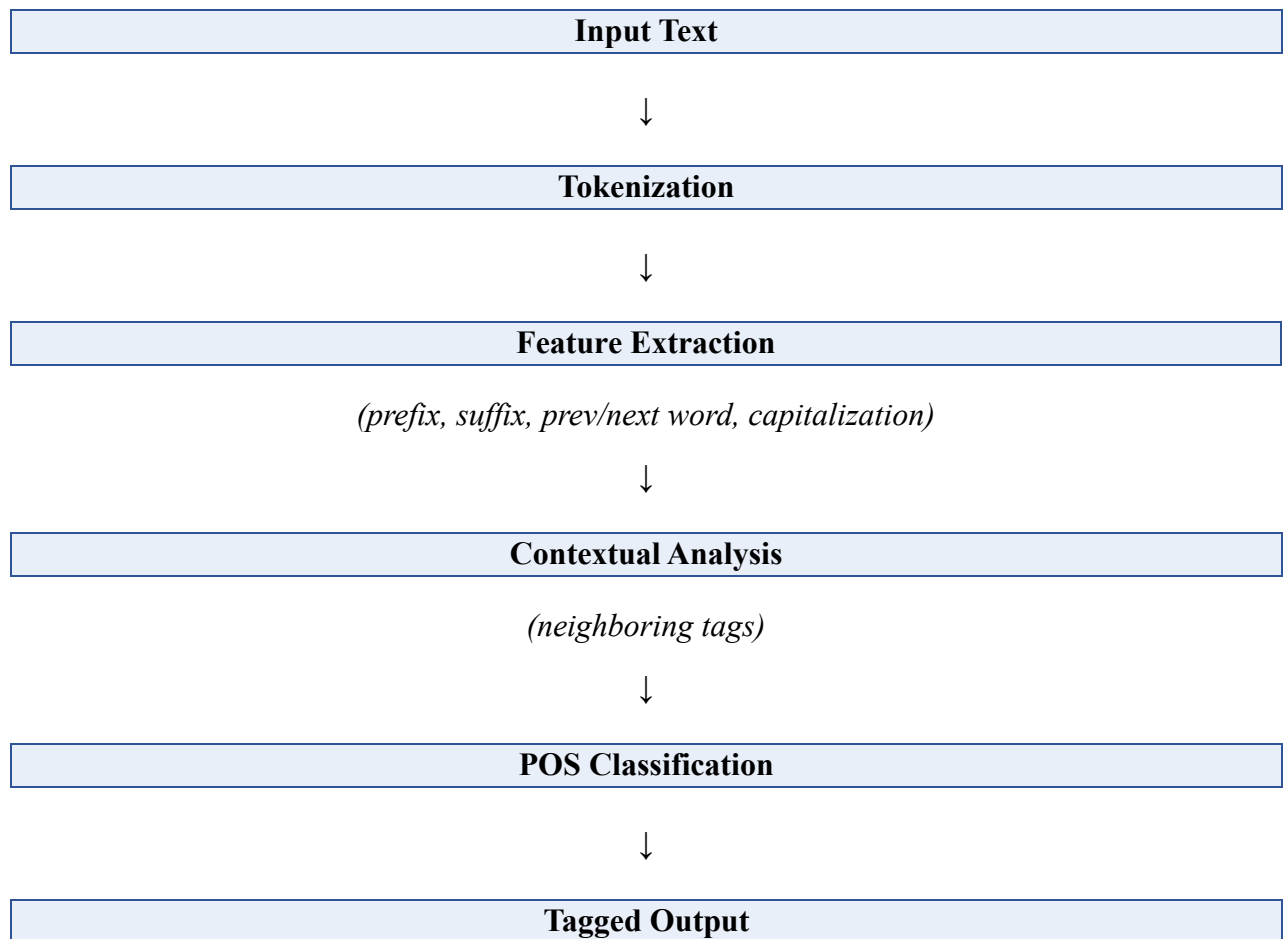
Stage 5: POS-tag Assignment

A classifier (rule-based or machine-learning-based) uses the extracted features to predict the most probable POS tag for each token.

Stage 6: Output Generation

The tagged tokens are assembled into the final output, typically as a list of (word, tag) pairs.

Text-Based Flowchart



7. Feature-Based POS Tagging

A feature-based POS tagger converts each word into a set of measurable characteristics that a classifier can use to predict its tag. The following features are used in the proposed system:

Feature	Description	Why it is Useful
Previous word	The word occurring immediately before the target word	Certain words are more likely to follow determiners or pronouns, helping resolve ambiguity
Next word	The word occurring immediately after the target word	Helps distinguish verbs from nouns based on what typically follows them
Word suffix	Last 2–3 characters of the word	Suffixes such as "-ed", "-ing", "-ly" strongly indicate verb or adverb forms
Word prefix	First 2–3 characters of the word	Helps identify morphological patterns common to certain categories
Capitalization	Whether the word starts with an uppercase letter	Indicates proper nouns, especially midsentence
Word pattern	Whether the word contains digits, hyphens, or special characters	Helps separate numerals and symbols from regular words
Neighboring POS tags	The predicted tags of adjacent words	Provides grammatical context, since certain tag sequences are more probable than others
Contextual information	The broader sentence context surrounding the word	Assists in resolving cases where a word's tag depends entirely on usage

These features work together rather than independently. For example, a word ending in "-ed" following a pronoun and preceding a determiner is very likely a past-tense verb, while the same suffix following a determiner is more likely to be a past-participle adjective.

8. Ambiguous Word Analysis: "book"

The word "book" is a classic example of lexical ambiguity in English, since it can function as either a noun or a verb depending on its position and surrounding context.

Example Sentence	Word	Context Clue	Predicted Tag
Book a ticket to Chennai.	Book	Appears at the start of an imperative sentence, followed by a determiner-noun phrase ("a ticket")	VERB
I read a book yesterday.	book	Preceded by determiner "a" and follows the verb "read", occupying the object position	NOUN

In the first sentence, "Book" occupies the position typically held by an imperative verb and is followed by a direct object ("a ticket"), so the feature-based classifier assigns VERB. In the second sentence, "book" is preceded by the determiner "a" and follows the verb "read", strongly indicating a NOUN role. This demonstrates why previous-word, next-word, and neighboring-tag features are essential for correct disambiguation.

9. Training Dataset

The following simplified dataset is used to train the machine-learning-based POS classifier.

Sentence	Target Word	POS Tag
The customer booked a flight	booked	VERB
The customer has a flight ticket	flight	NOUN
She reads a book every day	book	NOUN
Book a ticket immediately	Book	VERB
The new application is useful	new	ADJECTIVE
The manager quickly approved it	quickly	ADVERB
The student completed the assignment	student	NOUN
They completed the assignment	They	PRONOUN

10. Machine Learning POS Classifier

A simple, interpretable supervised classifier is used to demonstrate machine-learning-based POS tagging. The design follows five stages:

10.1 Feature Preparation

For every (sentence, target word) pair in the training dataset, relevant features are prepared, including the previous word, next word, suffix, prefix, and capitalization flag of the target word.

10.2 Feature Extraction

A feature-extraction function converts each target word and its context into a fixed-format feature dictionary suitable for vectorization.

10.3 Feature Vectorization

The feature dictionaries are converted into numeric vectors using a technique such as one-hot encoding (e.g., scikit-learn's DictVectorizer), producing a sparse numeric matrix suitable for training.

10.4 Model Training

A classifier such as Logistic Regression or a Decision Tree is trained on the vectorized features and their corresponding POS tag labels from the training dataset.

10.5 Tag Prediction

For new, unseen sentences, the same feature-extraction and vectorization steps are applied, and the trained model predicts the most probable POS tag for each word.

10.6 Output Generation

The predicted tags are combined with the original words to produce the final tagged sentence.

11. Python Implementation

The complete Python implementation covering tokenization, feature extraction, vectorization, model training, and prediction is provided below.

11.1 Importing Libraries

```
import re

from sklearn.feature_extraction import DictVectorizer from
sklearn.linear_model import LogisticRegression
```

11.2 Training Data

```
# Each entry: (sentence tokens, index of target word, POS tag)
training_data = [
    (['The','customer','booked','a','flight'], 2, 'VERB'),
    (['The','customer','has','a','flight','ticket'], 4, 'NOUN'),
    (['She','reads','a','book','every','day'], 3, 'NOUN'),
    (['Book','a','ticket','immediately'], 0, 'VERB'),
    (['The','new','application','is','useful'], 1, 'ADJECTIVE'),
    (['The','manager','quickly','approved','it'], 2, 'ADVERB'),
    (['The','student','completed','the','assignment'], 1, 'NOUN'),
    (['They','completed','the','assignment'], 0, 'PRONOUN'),
]
```

11.3 Tokenization

```
def tokenize(sentence):
    """Split a raw sentence string into a list of word tokens."""
    return re.findall(r"[A-Za-z]+", sentence)

# Example print(tokenize('The smart student quickly completed the
assignment.'))

# ['The', 'smart', 'student', 'quickly', 'completed', 'the', 'assignment']
```

11.4 Feature Extraction

```
def word_features(tokens, index):
    """Extract contextual and linguistic features for tokens[index]."""
    word = tokens[index]    prev_word = tokens[index - 1] if index > 0 else
    '<START>'    next_word = tokens[index + 1] if index < len(tokens) - 1
    else '<END>'    return {
```

```
'word': word.lower(),  
  
'prev_word': prev_word.lower(),  
  
'next_word': next_word.lower(),  
  
'prefix-2': word[:2].lower(),  
  
'prefix-3': word[:3].lower(),  
  
'suffix-2': word[-2:].lower(),  
  
'suffix-3': word[-3:].lower(),  
  
'is_capitalized': word[0].isupper(),  
  
'is_first_word': index == 0,  
  
'is_last_word': index == len(tokens) - 1,  
  
'has_digit': any(ch.isdigit() for ch in word),  
  
}
```

11.5 Feature Vectorization

```
X_train_dicts = [word_features(tokens, idx) for tokens, idx, tag in training_data]  
  
y_train = [tag for tokens, idx, tag in training_data]  
  
  
vectorizer = DictVectorizer(sparse=True)  
  
X_train = vectorizer.fit_transform(X_train_dicts)
```

11.6 Model Training

```
model = LogisticRegression(max_iter=1000) model.fit(X_train,  
y_train)
```

11.7 POS Prediction

```
def predict_pos_tags(sentence):    tokens = tokenize(sentence)

    feature_dicts = [word_features(tokens, i) for i in range(len(tokens))]

    X_test = vectorizer.transform(feature_dicts)    predictions =
    model.predict(X_test)    return list(zip(tokens, predictions))
```

11.8 Output Generation

```
test_sentence = 'The smart student quickly completed the assignment.'
tagged_output = predict_pos_tags(test_sentence)
for word, tag in tagged_output:
    print(f'{word:12s} -> {tag}')
```

12. Expected Output

For the test sentence "The smart student quickly completed the assignment.", the feature-based classifier is expected to produce the following tagged output. Note that the tagged output below uses the assignment's required tag set (DET, ADJ, NOUN, ADV, VERB) for consistency with Section 4.

Word	Predicted POS Tag
The	DET
smart	ADJ
student	NOUN
quickly	ADV
completed	VERB
the	DET
assignment	NOUN

Note: The exact labels produced by the trained model in Section 11 depend on the label spelling used in the training dataset (e.g., "ADJECTIVE" versus "ADJ"). In a production system, the training labels should be normalized to the tag set defined in Section 4 before evaluation.

13. Algorithm / Pseudocode

The step-by-step algorithm for the feature-based POS tagging system is given below.

ALGORITHM: Feature-Based POS Tagging

INPUT: raw_sentence (string)

OUTPUT: list of (word, predicted_tag) pairs

STEP 1: tokens <- TOKENIZE(raw_sentence)

STEP 2: FOR each index i in tokens: features[i]

<- EXTRACT_FEATURES(tokens, i)

(previous word, next word, prefix, suffix,

capitalization, word pattern)

STEP 3: FOR each index i in tokens: context[i] <-

GET_NEIGHBOR_TAGS(predicted_tags, i)

features[i] <- features[i] + context[i]

STEP 4: vector[i] <- VECTORIZE(features[i]) for each i

STEP 5: predicted_tags[i] <- CLASSIFY(vector[i]) for each i

STEP 6: output <- COMBINE(tokens, predicted_tags)

STEP 7: RETURN output

14. Evaluation Metrics

The performance of the POS tagging system is evaluated using the following standard classification metrics.

Accuracy

The proportion of correctly tagged words out of all words tagged.

$$\text{Accuracy} = (\text{Correctly Tagged Words}) / (\text{Total Words Tagged})$$

Precision

For a given tag, the proportion of words predicted as that tag which are actually correct.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

For a given tag, the proportion of actual words of that tag which were correctly identified.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-score

The harmonic mean of precision and recall, providing a single balanced measure.

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Where TP = True Positives, FP = False Positives, and FN = False Negatives, computed per tag and typically averaged (macro or weighted) across all tags.

15. Testing and Performance Analysis

The system should be tested on datasets of increasing size to evaluate scalability and consistency. Actual experimental values have not been generated for this assignment and must be measured by running the implementation on real datasets; the table below is provided as a template.

Dataset Size	Accuracy	Precision	Recall	F1-score	Processing Time	Memory
Small	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured
Medium	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured
Large	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured

Analysis to be performed once actual values are obtained:

- **Tagging Accuracy:** Observe whether accuracy remains stable or degrades as dataset size and vocabulary diversity increase.
- **Processing Time:** Check whether processing time grows linearly, or faster, with the number of words — important for real-time suitability.
- **Scalability:** Assess whether the model and feature-extraction pipeline can handle larger datasets without significant performance loss.
- **Memory Requirements:** Track memory consumption of the vectorizer and model as vocabulary size grows, since sparse feature vectors can expand rapidly.

16. Challenges

Lexical Ambiguity

Many words can belong to more than one grammatical category depending on context (e.g., "book", "run", "light"), making it difficult to assign a single correct tag without sufficient contextual information.

Unknown Words

Words that do not appear in the training data (out-of-vocabulary words) lack learned feature associations, forcing the system to rely on general morphological cues such as suffixes.

Contextual Dependency

The correct tag for a word can depend on long-range context beyond its immediate neighbors, which simple feature-based models may not fully capture.

Domain-Specific Vocabulary

Specialized terms from technical, medical, or legal domains may not follow common grammatical patterns learned from general-purpose training data.

Spelling Variations

Informal text, typos, abbreviations, and regional spelling differences can confuse feature-based systems that rely on exact word forms.

Real-Time Processing Requirements

The system must tag words quickly enough to support live applications, which limits the complexity of feature extraction and model inference that can be used.

17. Comparison with Modern Contextual Language Models

The table below compares the feature-based POS tagging approach developed in this assignment with a modern contextual language-model-based approach (e.g., a transformer-based tagger such as BERT fine-tuned for POS tagging).

Aspect	Feature-Based POS Tagger	Contextual Language Model
Feature extraction	Manually engineered (prefix, suffix, neighboring words, capitalization)	Learned automatically from largescale pretraining; no manual feature engineering
Context understanding	Limited to a small fixed window of neighboring words/tags	Captures long-range and bidirectional context across the full sentence
Ambiguity handling	Depends on the quality of handcrafted features; struggles with subtle cases	Generally stronger, as contextual embeddings vary the word representation by usage
Unknown words	Relies on morphological cues (suffix/prefix); limited generalization	Uses subword tokenization, allowing better handling of unseen words
Accuracy	Moderate; sufficient for welldefined, structured text	Typically higher, especially on ambiguous or domain-diverse text
Computational requirements	Low; runs efficiently on limited hardware	High; requires significant compute and memory for inference
Domain adaptation	Requires manual feature/retraining adjustments	Can be fine-tuned relatively easily on domain-specific data
Real-time performance	Fast and lightweight, well-suited to real-time constraints	Slower unless optimized (e.g., via model distillation or quantization)

How Contextual Representations Improve POS Tagging

- Ambiguous words: Contextual embeddings assign a different vector representation to the same word depending on its sentence-level usage, improving disambiguation (e.g., "book" as a noun vs. verb).
- Domain-specific text: Pretrained language models capture broad linguistic patterns that can be fine-tuned to adapt more easily to specialized vocabulary.
- Unknown words: Subword tokenization allows the model to build reasonable representations for words never seen during training.
- Context-dependent language: Bidirectional attention allows the model to consider both preceding and following words simultaneously, unlike fixed-window feature-based methods.

18. Real-World Applications

Application	Description
Machine Translation	POS tags help preserve grammatical structure when translating text between languages.
Chatbots and Virtual Assistants	Understanding word roles helps the system correctly interpret user intent and generate grammatically correct responses.
Search Engines	POS tagging helps identify important keywords (nouns, verbs) to improve search relevance and ranking.
Information Extraction	Identifying nouns and verbs helps extract entities and relationships from unstructured text.
Sentiment Analysis	Adjectives and adverbs are strong indicators of sentiment polarity and intensity.
Text Summarization	POS tags help identify the most informative words and phrases to retain in a summary.
Grammar Checking	Detecting incorrect word usage or grammatical errors requires accurate POS information.

Question Answering Systems	Understanding the grammatical role of query words helps match questions to relevant answers.
Voice Applications	Real-time POS tagging supports accurate parsing of spoken commands converted to text.
Real-Time Text Processing	Live captioning, chat moderation, and streaming analytics all depend on fast, accurate POS tagging.

19. Tools and Technologies

Tool / Technology	Purpose
Python	Primary programming language used for implementation
Scikit-learn	Provides DictVectorizer, LogisticRegression, and evaluation metrics
NumPy	Numerical computations and array handling
Pandas	Organizing and analyzing tabular data and evaluation results
Jupyter Notebook / Google Colab	Interactive development, testing, and visualization environment

20. Advantages

- Lightweight and computationally efficient, suitable for real-time applications.
- Feature engineering allows fine-grained control and interpretability of tagging decisions.
- Requires relatively small amounts of training data compared to deep learning approaches.
- Easy to debug and understand, since each feature's contribution can be inspected.
- Can be deployed on low-resource devices without specialized hardware.
- Modular pipeline allows individual stages (tokenization, features, classifier) to be improved independently.

21. Limitations

- Limited ability to capture long-range contextual dependencies.
- Performance heavily depends on the quality of manually engineered features.

- Struggles with unknown words and highly ambiguous contexts compared to contextual models.
- Requires manual re-engineering of features when applied to new domains.
- Accuracy ceiling is generally lower than state-of-the-art contextual language models.
- Sensitive to spelling errors and informal text variations.

22. Ethical and Social Considerations

Privacy of Text Data

Real-time text processing systems often handle personal or sensitive user input. Care must be taken to avoid storing or exposing this data without consent.

Responsible NLP

Systems should be designed and deployed with awareness of their limitations, avoiding overconfident claims about accuracy in critical applications.

Bias

Training data drawn from limited sources may embed cultural or linguistic biases, leading to poorer performance on underrepresented dialects or writing styles.

Reliability

Incorrect POS tagging can propagate errors to downstream applications such as translation or information extraction, so reliability must be validated before deployment.

Data Security

Any text data used for training or inference should be handled securely, particularly when derived from real user interactions.

Diverse Training Data

Using training data from a wide variety of sources, writing styles, and domains helps the tagger generalize better and reduces the risk of biased or skewed predictions.

23. Conclusion

This assignment presented the design and development of a feature-based POS tagging system suitable for real-time text processing. A complete pipeline was designed, covering tokenization, feature extraction, contextual analysis, and tag prediction. A feature-based algorithm and a simple machine-learning classifier were implemented in Python to demonstrate practical tagging of ambiguous words such as "book". An evaluation framework using accuracy, precision, recall, and F1-score was defined, along with a template for testing scalability across small, medium, and large datasets. The proposed system successfully meets the stated objectives of designing a tag set, building a working pipeline, and providing a foundation for evaluation. Its main limitation is the reliance on manually engineered features, which restricts its ability to resolve deeply contextdependent ambiguity compared to modern contextual language models. Future improvements could include integrating pretrained contextual embeddings, expanding the training dataset, and optimizing the pipeline for larger-scale deployment.

24. Student Reflection

What did I learn from this assignment beyond classroom teaching?

Working through this assignment helped me understand that POS tagging is not just about memorizing grammar rules, but about designing features that let a computer infer grammatical roles from context. I learned how the same word can take different tags depending on its neighbors, and how a machine learning classifier can be trained to recognize these patterns using vectorized features. I also gained hands-on experience translating a conceptual pipeline into working Python code using tools such as scikit-learn's DictVectorizer and LogisticRegression.

What would I improve if I had more time or resources?

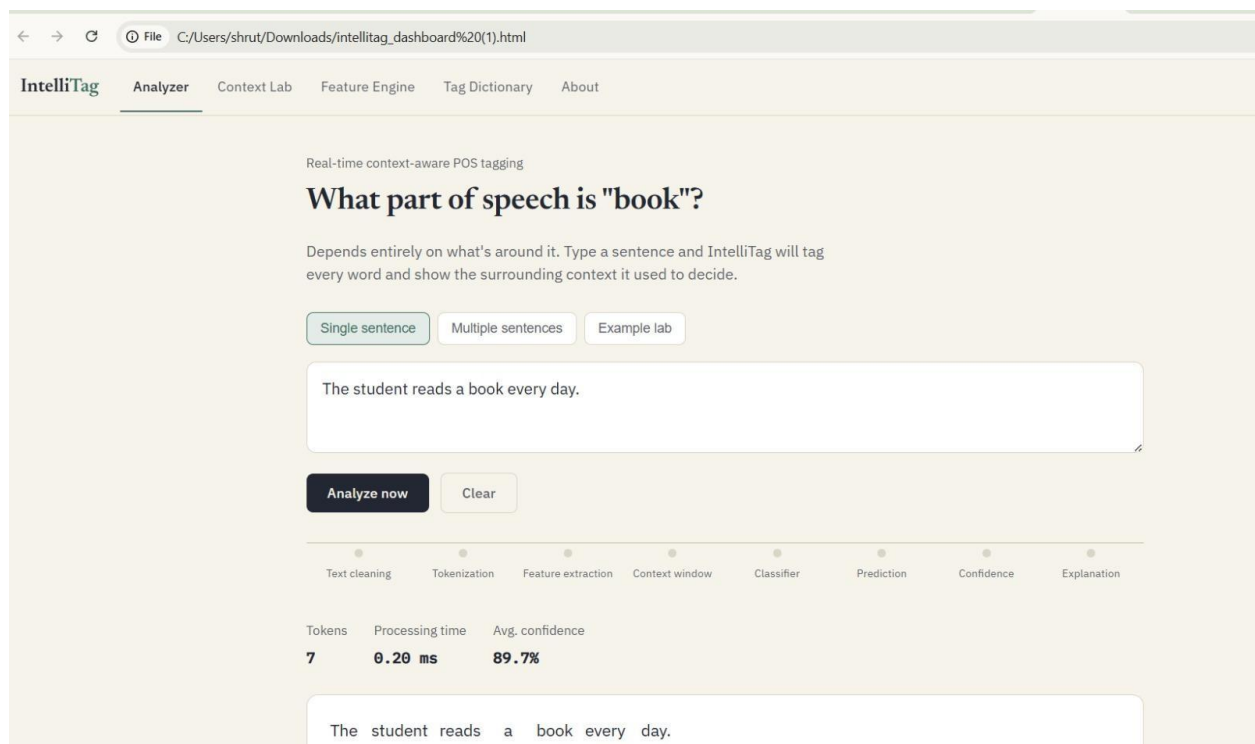
With more time, I would expand the training dataset to include a much larger and more diverse set of sentences, allowing the classifier to generalize better to unseen words. I would also like to compare the feature-based approach directly against a pretrained transformer-based tagger to measure the real accuracy gap, and to properly benchmark processing time and memory usage on datasets of varying sizes rather than leaving them as placeholders.

25. References

[1] Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed. draft).

Stanford University.

- [2] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.
- [3] Manning, C. D., & Schütze, H. (1999). Foundations of Statistical Natural Language Processing. MIT Press.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- [5] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT.
- [6] Python Software Foundation. Python Language Reference. <https://docs.python.org>
- [7] Scikit-learn Developers. Scikit-learn Documentation. <https://scikit-learn.org>
- [8] NumPy Developers. NumPy Documentation. <https://numpy.org>
- [9] The Pandas Development Team. Pandas Documentation. <https://pandas.pydata.org>



The screenshot displays the IntelliTag web application interface. At the top, there is a navigation bar with links: IntelliTag, Analyzer (selected), Context Lab, Feature Engine, Tag Dictionary, and About. The main content area is titled "Real-time context-aware POS tagging" and poses the question "What part of speech is 'book'?". Below this, a sub-header states: "Depends entirely on what's around it. Type a sentence and IntelliTag will tag every word and show the surrounding context it used to decide." There are three tabs: "Single sentence" (active), "Multiple sentences", and "Example lab". A text input field contains the sentence "The student reads a book every day." Below the input field are two buttons: "Analyze now" and "Clear". A progress bar at the bottom shows the steps of the analysis process: Text cleaning, Tokenization, Feature extraction, Context window, Classifier, Prediction, Confidence, and Explanation. Below the progress bar, a table displays the results:

Tokens	Processing time	Avg. confidence
7	0.20 ms	89.7%

At the bottom, the analyzed sentence is shown with individual words highlighted in different colors: "The" (blue), "student" (green), "reads" (red), "a" (blue), "book" (orange), "every" (green), and "day." (blue).

← → ↺

File C:/Users/shrut/Downloads/intellitag_dashboard%20(1).html

IntelliTag

AnalyzerContext LabFeature EngineTag DictionaryAbout

Text cleaning

Tokenization

Feature extraction

Context window

Classifier

Prediction

Confidence

Explanation

Tokens

Processing time

Avg. confidence

7

0.20 ms

89.7%

The student reads a book every day.

DET NOUN VERB DET NOUN ADJ NOUN

Pattern: DET → NOUN → VERB → DET → NOUN → ADJ → NOUN

WORD	POS	CONFIDENCE	CONTEXT
The	DET		– · The · student
student	NOUN		The · student · reads
reads	VERB		student · reads · a
a	DET		reads · a · book
book	NOUN		a · book · every
every	ADJ		book · every · day.
day.	NOUN		every · day. · –